

Unit-02

Output Primitives

2.1 Pixel and Straight Line

Pixel:

A pixel (short for "picture element") is the smallest unit of a digital image or display. It is a single point in a raster image and is the building block of any image shown on a screen.

Properties of a Pixel:

- Position: Defined by its coordinates in a 2D grid (x, y).
- Colour: Each pixel can have a colour defined by its RGB (Red, Green, Blue) values or other colour models.
- Size: In modern displays, pixels are very small, and their size depends on the screen resolution.
- Role in Computer Graphics: A pixel represents a part of an image, and the arrangement of pixels creates the complete visual representation on screens, from simple graphics to complex images and videos.

Straight Line in Computer Graphics:

A line is a collection of points arranged in a straight path. In computer graphics, a line is represented by its endpoints: the start points and the end point.

Equation of a Line:

In a 2D Cartesian coordinate system, a line can be represented using its slope (m) and y-intercept (b) in the slope-intercept form:

$$y = mx + b$$

Line Drawing Problem:

The challenge in computer graphics is to efficiently draw a straight line between two points on a screen (represented by pixels).

2.2 Line Drawing Algorithms: DDA, Bresenham's Line Drawing Algorithm

Digital Differential Analyzer (DDA) Algorithm:

DDA

Definition:

DDA is a simple and widely used algorithm to draw a line between two points in computer graphics.

Working Principle:

DDA computes intermediate points along the line based on the slope (m) between the starting and ending points. The basic idea is to compute either the x or y value incrementally and then calculate the other using the line equation.

Steps:

1. Input: Two points $P_1(x_1, y_1)$ and $P_2(x_2, y_2)$.

2. Calculate differences:

$$dx = x_2 - x_1,$$

$$dy = y_2 - y_1$$

3. Calculate number of steps:

$$\text{steps} = \max(\text{abs}(dx), \text{abs}(dy))$$

4. Calculate increments:

$$\text{Delta_x} = dx / \text{steps},$$

$$\text{Delta_y} = dy / \text{steps}$$

5. Iterate:

Starting from P_1 , compute subsequent points:

$$x = x_1 + n * \text{Delta_x},$$

$$y = y_1 + n * \text{Delta_y}$$

6. Plot points: Plot the calculated points (x, y) for each iteration.

Advantages:

- Simple to understand and implement.
- Suitable for drawing lines with arbitrary slopes.

Disadvantages:

- Floating-point operations make it slower and less efficient compared to integer-based algorithms like Bresenham's.

Bresenham's Line Drawing Algorithm:

Definition:

Bresenham's algorithm is an efficient way of drawing straight lines using only integer addition and subtraction.

Working Principle:

The algorithm uses decision variables to decide which pixel to plot next based on the error term (deviation from the ideal line path).

Steps:

1. Input: Two points $P_1(x_1, y_1)$ and $P_2(x_2, y_2)$.
2. Calculate differences: $dx = x_2 - x_1$, $dy = y_2 - y_1$
3. Initialize decision parameter:

$$p = 2dy - dx$$

4. Plot initial point: Plot $P_1(x_1, y_1)$.

5. Iterate:

If $p < 0$, plot $(x+1, y)$ and update $p = p + 2dy$.

If $p \geq 0$, plot $(x+1, y+1)$ and update $p = p + 2(dy - dx)$.

6. Repeat:

- Continue this process until P_2 is reached.

Advantages:

- More efficient than DDA because it uses only integer calculations.

- Faster and more suitable for raster devices.

Disadvantages:

- More complex to understand and implement compared to DDA.

Digital Differential Analyzer (DDA) Algorithm

Definition:

The Digital Differential Analyzer (DDA) algorithm is a simple and efficient line-drawing algorithm in computer graphics. It is used to draw a straight line between two points on a screen by calculating intermediate points along the line.

Working Principle:

DDA incrementally computes points along a line based on its slope (m) and then plots them. It involves floating-point arithmetic to calculate the next point either in the x-direction or y-direction.

Steps to Implement DDA Algorithm:

1. Input

- Start Point: (x_1, y_1)
- End Point: (x_2, y_2)

2. Calculate the differences:

$$dx = x_2 - x_1,$$

$$dy = y_2 - y_1$$

3. Find the number of steps required:

- Steps are the larger of $|dx|$ or $|dy|$:

$$\text{steps} = \max(|dx|, |dy|)$$

4. Calculate increments per step:

- Increment values for x and y are computed as:

$$\text{delta_x} = dx / \text{steps},$$

$$\text{delta_y} = \text{dy} / \text{steps}$$

5. Iterate through steps to compute points:

- Starting from the initial point (x1, y1), calculate subsequent points:

$$x = x_1 + n * \text{delta_x},$$

$$y = y_1 + n * \text{delta_y}$$

where $n = 0, 1, 2, \dots, \text{steps}$.

6. Plot the calculated points.

Advantages of DDA Algorithm:

- Simple and easy to implement.
- Handles lines with arbitrary slopes.

Disadvantages of DDA Algorithm:

- Uses floating-point calculations, which are slower than integer arithmetic.
- Rounding operations can introduce errors in some cases

2.3 Midpoint Circle Algorithm Definition:

The Midpoint Circle Algorithm is a technique for drawing circles on raster displays. It calculates the points along the circumference of a circle incrementally based on the midpoint decision method.

Steps:

1. Input: The centre of the circle (h, k) and the radius r.

2. Initial Values:

Start at the point (x₀, y₀) where x₀ = 0 and y₀ = r.

The initial decision parameter: p₀ = 1 - r

3. Plot Symmetric Points:

For each point, plot the eight symmetric points of the circle:

$(h+x, k+y)$, $(h-x, k+y)$, $(h+x, k-y)$, $(h-x, k-y)$,

$(h+y, k+x)$, $(h-y, k+x)$, $(h+y, k-x)$, $(h-y, k-x)$

4. Update Decision Parameter:

If $p < 0$, move horizontally to the right; update $p = p + 2x + 3$.

Otherwise, move diagonally down-right; update $p = p + 2x - 2y + 5$.

5. Repeat:

- Continue plotting points and updating the decision parameter until the entire circle is drawn.

Advantages:

- Efficient for drawing circles using integer calculations.
- Simple and accurate for raster graphics.

Disadvantages:

- Applicable only for circles (not other shapes like ellipses).

Important Formulas:

DDA Algorithm:

- Step count:
$$\text{steps} = \max(\text{abs}(\text{dx}), \text{abs}(\text{dy}))$$
- Increments:
$$\text{Delta_x} = \text{dx} / \text{steps},$$
$$\text{Delta_y} = \text{dy} / \text{steps}$$

Bresenham's Line Algorithm:

- Initial decision parameter: $p = 2dy - dx$

Midpoint Circle Algorithm:

- Initial decision parameter: $p_0 = 1 - r$